Persistent Agent Context with Progressive Retrieval Attention: Typed Resource Discovery Before Neural Materialization

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 25, 2026

Abstract

Agent runtimes often place every tool definition in every prompt before the model has decided which capability it needs. We separate this process into typed discovery, bounded disclosure, model-side call construction, and host-authorized execution. M0 resolves opaque catalogs up to 8,192 resources; M1 shows causal use of one selected native-K/V definition in a toy decoder. With frozen Qwen3-0.6B, M2 obtains 20/20 exact calls from selected or oracle schemas, versus 0/20 from shuffled, irrelevant, or empty disclosure and 17/20 from all 18 tools. M3 safely executes 20/20 selected calls in an in-memory harness and preserves every result as a typed observation; 17/20 continuations include every returned value. M4 completes 10/15 three-to-five-step workflows with reactive just-in-time disclosure, versus 3/15 with all required tools visible and 0/15 without refresh. M5 then falsifies the proposed speculative extension: a metadata/schema graph raises required-tool recall to .933 but obtains 0/15 task success under static disclosure. M6 also stops negatively: indexed lexical discovery reaches 1.000 single-step Top-1 and token routing reaches .889 multi-step required-tool recall, while native mean K, full token QK, and zero-shot Paper-2.8 projections are worse. M6.5 then tests 306 authored canonical, paraphrastic, colloquial, implicit, contextual, and multilingual requests. On 144 frozen test rows, BM25 falls to .347 Top-1; typed tags reach .743 and a dictionary/embedding hybrid reaches .729. A validation-calibrated staged resolver selects on .597 of requests at .907 selective accuracy. M7 reevaluates frozen native geometry on the same rows: native mean K, full token QK, and two transferred compressors obtain only .049–.063 Top-1. The supported path is therefore model-independent semantic discovery followed by reactive disclosure; graph speculation and native-Q/K discovery remain bounded experimental modes.

1 The Agent Catalog Problem

An append-only agent prompt couples the size of the logical environment to the transformer's active context. Tool schemas, skills, system instructions, artifacts, and old observations are repeatedly exposed even when a turn uses only one of them. Compaction can reduce prompt length, but it may erase provenance or stale-versus-current distinctions.

PRA suggests a different decomposition:

$$\text{agent state} = \text{bounded native workspace} + \text{persistent addressable resources}.$$

This decomposition does not make storage free. Source text, indexes, cached representations, resident K/V, transferred K/V, and attention-visible K/V are different costs. It instead introduces a concrete question before materialization: which resource identity should be resolved, by which discovery mechanism, and with enough confidence to act?

This paper reports a staged sequence of mechanism gates. M0 asks whether a typed resolver can preserve selection quality as a catalog grows, whether policy hints reduce unnecessary search, and whether confidence prevents an accurate ranker from becoming an unsafe selector. M1 asks whether a small decoder causally uses one selected definition after it is encoded as native K/V. M2–M4 add a frozen pretrained model, typed execution, and multi-step reactive discovery. M5 asks whether capability neighborhoods should be disclosed before planning. M6 asks whether model-native query/key state improves resource discovery enough to justify a tighter deployment boundary. M6.5 asks how far an external semantic resolver can go when canonical lexical overlap disappears, and M7 repeats the native comparison on exactly that frozen semantic-hard test set. Persistent- session recall, production speed at large catalog sizes, and external side effects remain outside these gates.

2 Typed, Versioned Resources

A resource has a stable typed identity, versioned source, aliases, schema, side-effect class, tenant, and revocation state. Representative identifiers are

```
!!ref:tool:calendar:create_event:v3!!
!!ref:skill:paper-review:v2!!
!!ref:artifact:experiment-17!!
!!ref:session:parent/decision-14!!
```

Identity is distinct from text, index entries, semantic summaries, cached K/V, and current residency. A cache fingerprint therefore includes source/version, model, tokenizer, routing configuration, and any position-sensitive encoding state.

The implementation in `src/pra_hf/agent_resources.py` enforces tenant, revocation, and allowed-URI filters before ranking. Discovery returns a trace, not only a winner: the executed channel path, per-candidate provenance, confidence, fallback count, and final select/ask/abstain decision remain available to the caller. The resolver never authorizes or executes a tool.

3 Discovery Channels and Policy

The SDK exposes

$$\pi \in \{\text{auto}, \text{explicit}, \text{token}, \text{index}, \text{semantic}, \text{hybrid}, \text{adaptive}\}.$$

Hints can be attached to a request or inherited from a resource collection. Strict hints execute one channel. Non-strict hints specify a starting point and permit calibrated fallback.

Explicit. A typed URI is resolved by exact map lookup. This is deterministic after access checks.

Token. Names, aliases, descriptions, and token n-grams are compared lexically. The M0 implementation is deliberately exhaustive and therefore a correctness control, not a production latency path.

Index. Persistent exact, token-posting, n-gram, and BM25 structures narrow the candidate set before scoring. Index fingerprints make freshness auditable.

Semantic. M0 uses a declared 96-dimensional signed-hash control over the resource's action, object, and family concepts. It contains no resource ID. This idealized control isolates the policy mechanism; it is not evidence about a learned embedding model. M6.5 replaces that control with two deployable external families: typed operation/object concepts with source provenance, and compact local encoders whose tool vectors are precomputed at registration. Neither family calls the generation model or reads its hidden state.

Hybrid and adaptive. Hybrid fuses all available evidence in one expensive stage. Adaptive begins cheaply and escalates only when calibrated confidence is insufficient. A representative ladder is

$$\text{explicit} \to \text{token} \to \text{index} \to \text{semantic} \to \text{hybrid} \to \text{ask/abstain}.$$

The concrete path is recorded per request.

3.1 Confidence is part of correctness

For candidate r_i, the calibrator estimates

$$c_i \approx P(r_i \text{ is intended} \mid q, \mathbf{s}_i, p_i),$$

where $\mathbf{s}_i$ denotes channel scores and p_i their provenance. High confidence permits selection; intermediate confidence asks or searches more; low confidence abstains. For a positive request, the actionable outcome is correct only when the right resource is ranked first *and* the decision is `select`. For an ambiguous or nonexistent request, it is correct only when the decision is `ask` or `abstain`. This definition prevents a low-confidence ranker from receiving operational credit merely because its latent top candidate was right.

3.2 Discovery and disclosure are different controls

Discovery answers how candidates are found: explicit URI, token matching, persistent index, semantic score, or a calibrated combination. Disclosure answers which bounded capability set becomes visible to the model. A request can therefore pair indexed discovery with minimal disclosure, or explicit discovery with a broader planning neighborhood. The SDK represents this as

$$\text{AgentResourcePolicy} = (\text{DiscoveryPolicy}, \text{ToolDisclosurePolicy}),$$

rather than overloading one `top_k`. Resolve mode targets one identity $q \mapsto r^*$. Explore mode targets a set $(q, state) \mapsto R_K$ and requires recall, precision, unsafe exposure, plan coverage, and task-success metrics. Neither form authorizes execution.

The M5 implementation builds a typed capability graph from category, object, API and operation families, curated tags, deterministic description keywords, and directional producer–consumer schema edges. Every disclosed identity retains its root, edge type, depth, weight, and inclusion source. Destructive neighbors are suppressed by default. Fixed `minimal`, `local`, `planning`, `broad`, and `adaptive` profiles bound each signal independently.

4 M0 Experimental Design

4.1 Opaque catalogs

The generator in `src/data/agent_resources.py` creates identifiers such as `tool_zaf_193`. Each resource combines held-out action, object, and family concepts with aliases, a schema, a version, and

a side-effect class. The opaque names prevent pretrained knowledge from solving the benchmark. Catalogs contain 8, 32, 128, 512, 2,048, and 8,192 resources.

Each held-out identity produces six positive requests: explicit URI, exact name, alias, typo, semantic paraphrase, and descriptive reference. Each split also contains one ambiguous and one nonexistent request. Validation identities calibrate thresholds; test identities are disjoint. At sizes 32–8,192, each policy is evaluated on 540 positive and 10 negative test requests across seeds $\{11, 23, 37, 53, 71\}$. The 8-resource condition uses 180 positive and 10 negative requests. The complete artifact contains 31,680 validation and test policy traces.

4.2 Policies and metrics

We compare five fixed policies (explicit, token, index, semantic, and hybrid), unguided `auto`, a request-level hint, and adaptive fallback. The oracle is the least-cost fixed policy that ranks the target first on that query. Reported metrics include top-1 accuracy, selection coverage, selective and actionable outcome accuracy, safe nonaction and false-action rates, retrieval stages, fallback count, Brier score, expected calibration error (ECE), quality regret, and policy-cost regret. Confidence thresholds are fitted only on the validation split. Intervals are percentile bootstrap intervals over five seed means; deterministic outcomes can consequently have zero-width intervals.

4.3 Systems accounting

Cold index construction and a full immutable rebuild after mutating one percent of resource versions are timed separately. Warm component latency is the mean of repeated CPU Python calls. Policy-path latency is reconstructed by summing the measured components actually executed by each trace. These timings exclude tokenization, a model forward pass, native-K/V encoding or transfer, attention, and tool execution. They characterize this reference implementation, not an optimized serving system.

5 Results

5.1 Selection scales, but policies differ operationally

Table 1 reports the largest catalog. Hinted and adaptive policies select every positive target and safely reject every negative request. Their equal quality hides a meaningful systems difference: hints reduce the mean path from 2.39 stages to 1.08 and the measured component estimate from 1,084 to 691 ms per request. The fixed hybrid ranker also attains 100% top-1, but calibration leaves 1.7% of positive requests unselected. Its exhaustive 1,405 ms component cost and 4.17 cost-regret units make it a poor default.

Table 1: Held-out M0 results at 8,192 resources. Top-1 is computed on 540 positive requests; outcome includes ten ambiguous/nonexistent controls. Time is reconstructed CPU Python discovery-component time.

Policy	Top-1	Coverage	Outcome	Safe neg.	Stages	ms/query
Explicit	.167	.167	.182	1.000	1.00	5
Token	.713	.537	.545	1.000	1.00	1,174
Index	.833	.000	.018	1.000	1.00	27
Semantic	.333	.000	.018	1.000	1.00	339
Hybrid	1.000	.983	.984	1.000	1.00	1,405
Auto	.833	.000	.018	1.000	1.00	24
User hint	**1.000**	**1.000**	**1.000**	**1.000**	1.08	691
Adaptive	**1.000**	**1.000**	**1.000**	**1.000**	2.39	1,084

The unguided automatic policy exposes why ranking is insufficient. It ranks explicit, name, alias, typo, and description requests correctly, misses the semantic-paraphrase stratum, and obtains .833 top-1. Calibration nevertheless causes it to ask on every positive request, so its actionable outcome accuracy is only $2/110 = .018$ per seed: the two safe negative decisions. Fixed index has the same operational pattern. Conversely, fixed token acts selectively and is always right when it acts, but covers only .537 of positive requests.

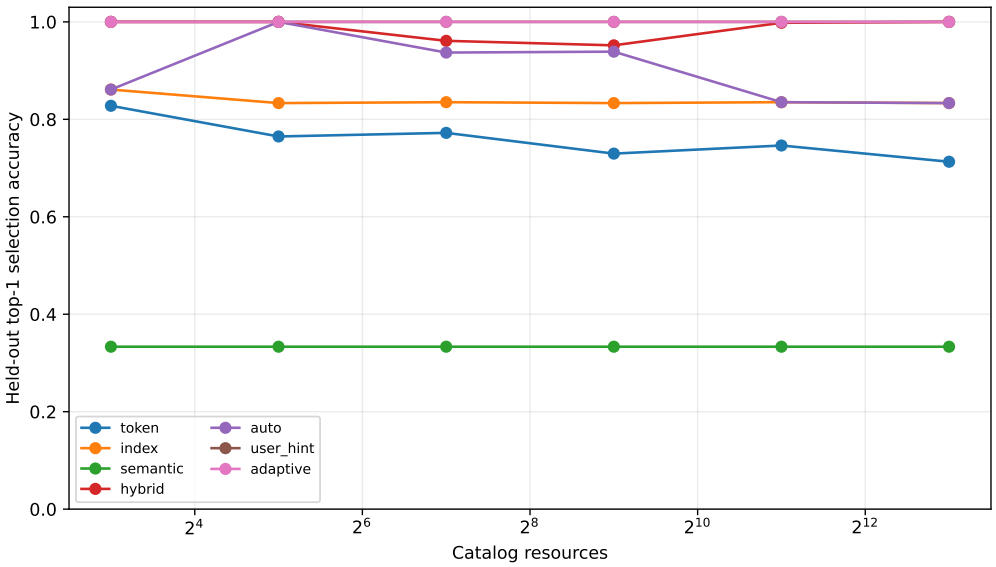

Figure 1: Positive-request top-1 accuracy as catalog size increases. Ranking alone does not encode whether confidence permits selection.

Figure 1 shows that hints and adaptive fallback maintain 1.0 top-1 at every tested size. The result is not uniformly safe at the smallest size: in the 8-resource condition each policy attains 1.0 positive top-1, but hinted and adaptive routing each falsely select the ambiguous held-out case in all five seeds. Their overall outcome accuracy is .974 and safe-nonaction rate is .5. From 32 resources onward, both negative cases are handled correctly in all seeds. This inversion is a calibration artifact of the controlled score distribution and cautions against treating larger catalogs as automatically harder along every axis.

5.2 No fixed channel is the oracle everywhere

At 8,192 resources, the least-cost correct oracle chooses explicit lookup for 16.7% of positive requests, indexed lookup for 66.7%, and semantic lookup for 16.7%. Neither token nor hybrid is ever the least-cost correct choice. User hints have zero quality regret and 0.083 cost-regret units, compared with zero quality regret and 0.5 cost regret for adaptive fallback. This supports a narrow claim: when callers know the reference form, passing that information avoids discovery work; adaptive escalation remains useful when they do not.

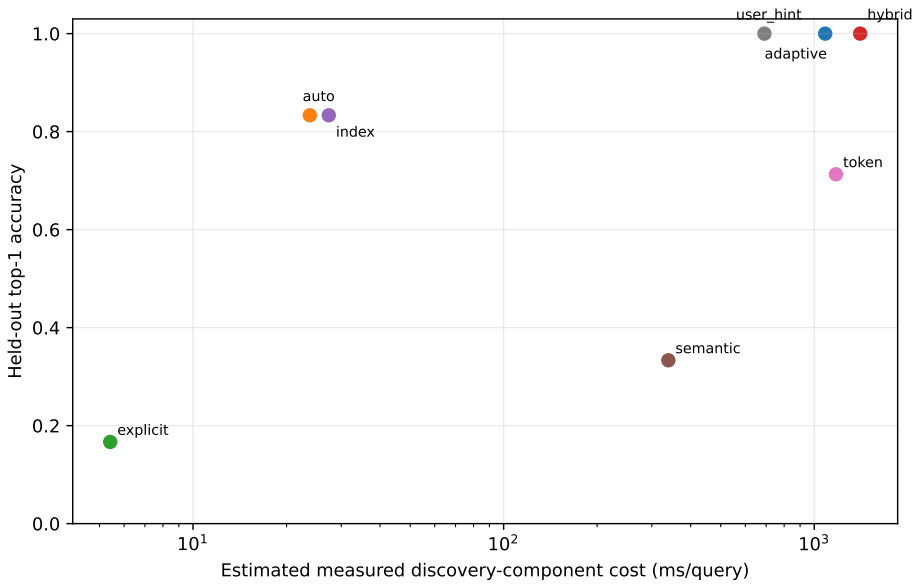

Figure 2: Largest-catalog ranking quality versus measured discovery-component cost. The log axis is needed because explicit and exhaustive hybrid paths differ by more than two orders of magnitude.

5.3 Logical catalog size is not active materialization

Every generated definition contains 18 accounting tokens. The largest catalog therefore contains 147,456 logical definition tokens, whereas resolving one resource requests 18 tokens, or 0.0122% of the catalog. Figure 3 makes that distinction visible. The number is an accounting upper bound on selected source text; M0 does not encode definitions into native K/V and therefore does not establish attention-memory or latency savings.

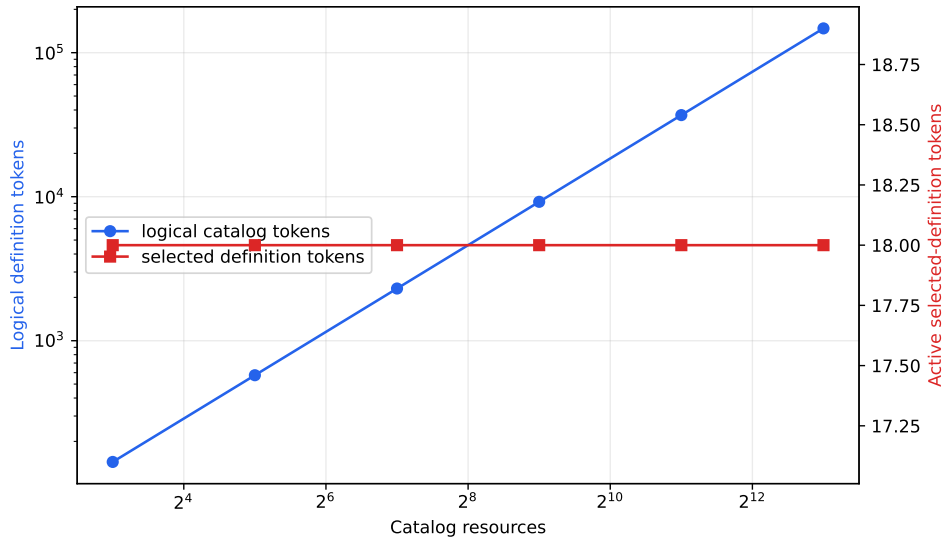

Figure 3: Logical catalog definitions grow with catalog size while the one-item selected-definition accounting remains at 18 tokens. The axes intentionally separate logical storage from active materialization.

5.4 Persistent indexing helps, but is not yet production-ready

At 8,192 resources, index construction takes 1.727 s, a full rebuild after a one-percent version mutation takes 1.753 s, and the estimated Python index payload is 5.27 MiB. Warm indexed lookup takes 27.4 ms and scores 966 candidates on average (11.8% of the catalog). By comparison, exhaustive semantic, token, and hybrid controls take 339, 1,174, and 1,405 ms. The index build amortizes to 44.6 ms/query after 100 uses and 29.1 ms after 1,000 uses.

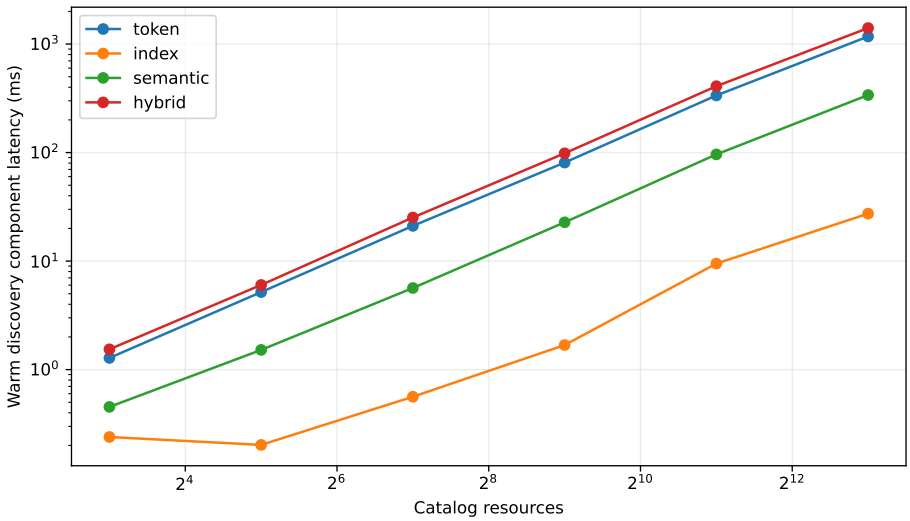

Figure 4: Warm CPU Python component latency. Exhaustive channels scale poorly; the indexed path narrows candidates but still touches about 11.8% at 8K.

These numbers establish an optimization target rather than a speed claim. Production token postings should avoid Python object traversal, semantic search should use an approximate vector index, unions should be deduplicated before scoring, and mutation should update affected postings rather than rebuild the entire immutable index. Cold/warm serving measurements must also include tokenization and downstream native-K/V work.

6 What the M0 Gate Establishes

The mechanism satisfies four narrow requirements. First, typed resources can be resolved without exposing the full catalog to a language model. Second, different reference forms favor different channels. Third, request hints can match adaptive quality with fewer stages. Fourth, action-aware calibration can distinguish a correct ranking from a decision safe enough to use.

M0 alone does *not* satisfy the paper's broader context gate. Its 18-token active count is not a measured model working set. There is no model-side argument generation, observation-conditioned continuation, persistent-history recall, skill composition, or subagent inheritance. Its negative controls are small and its semantic control is idealized. We therefore treat M0 as resolver evidence rather than evidence that PRA improves agents.

7 M1: A Causal Toy-Model Gate

7.1 Protocol separates discovery, generation, and execution

The synthetic language gives each tool an opaque identity and a compact definition. Semantics appear only in that definition. A held-out query states an action, object, family, and raw argument, but contains neither the resource identity nor one of four formatting schemas. The host first discovers a stable URI and binds it to a temporary slot such as `@0`. The model must emit `slot|schema|argument`; the host resolves the slot and accepts the call only when the selected URI, schema, and formatted argument are all exact. A deterministic observation is then supplied and the model must continue with the expected answer. No external side effect is performed.

This boundary matters. Discovery quality is measured by stable URI identity. The model is responsible for schema-conditioned call construction. The host is responsible for binding and validation. A correct retrieval cannot receive end-to-end credit for an incorrect call, and a memorized call string cannot receive selection credit for an incorrect resource.

7.2 Model, training, and causal conditions

The character-level decoder has 506,400 parameters, four layers, hidden width 96, feed-forward width 384, RoPE, and PRA native-K/V at layers 1 and 3. Each of five seeds trains for 3,000 steps with a mixture of native-memory examples (50%), directly presented selected definitions (30%), and eager eight-item catalogs (20%). Loss is applied only to the call and post-observation continuation. Evaluation uses eight examples per seed at catalog sizes 8, 32, and 128.

We compare oracle memory, idealized semantically discovered memory, shuffled memory, disabled memory, the selected definition directly in the native prompt, and the eager catalog. All conditions share model weights. Shuffling preserves memory presence and size but selects the wrong definition; disabling removes memory. Eager conditions that exceed the 512-token native window are recorded as context overflow and are not truncated into an easier task.

7.3 Selected native K/V is used causally

Table 2 reports the strict end-to-end metric: correct resource, exact call, and exact continuation. Discovered memory matches oracle memory at all three sizes. Shuffled and disabled conditions obtain zero, so performance cannot be explained by the query alone or by generic memory activation.

Table 2: M1 end-to-end success, averaged over five seeds and eight held-out examples per seed. A dash denotes native-window overflow rather than failure.

Catalog	Discovered	Oracle	Direct	Shuffled	Disabled	Eager
8	1.000	1.000	0.925	0.000	0.000	0.025
32	0.950	0.950	0.825	0.000	0.000	–
128	0.925	0.925	0.800	0.000	0.000	–

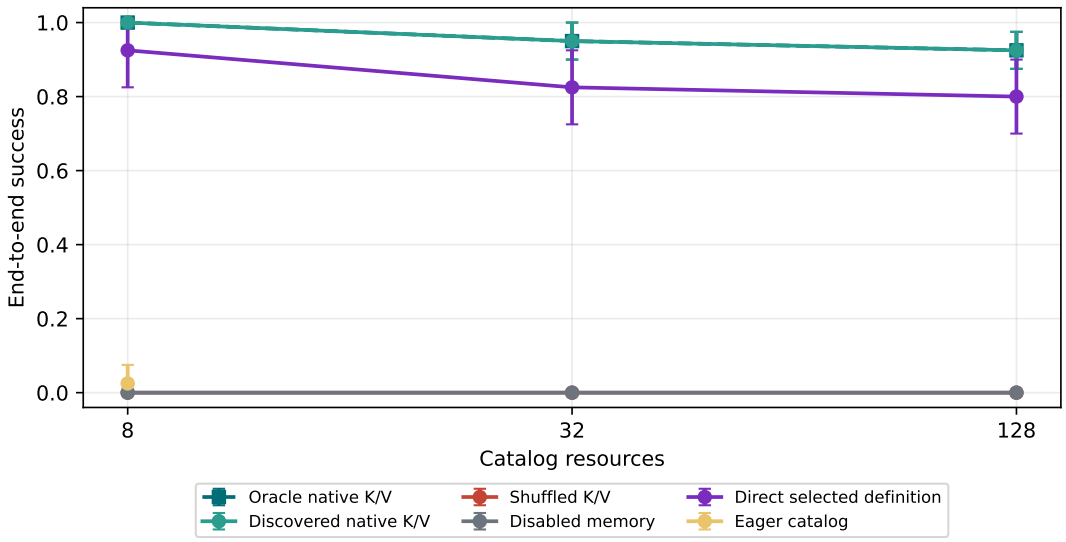

Figure 5: M1 end-to-end success. Oracle and discovered curves overlap; shuffled and disabled controls remain at zero. Eager evaluation is defined only where the complete catalog fits.

Every seed favors oracle memory over shuffled and disabled memory at every catalog size. With five paired seeds, however, the minimum attainable exact two-sided sign-flip value is $p = .0625$. Oracle and discovered memory have zero paired difference. Direct presentation is lower by .075 at size 8 and .125 at sizes 32 and 128, but those paired differences are not statistically decisive. We report the direction without claiming that PRA presentation is intrinsically better than direct presentation.

7.4 Logical growth is separated from active K/V

Figure 6 measures the neural working set that M0 could only account for. Logical catalog text grows with catalog size and crosses the native window before size 32. The selected native-K/V payload remains near one definition per PRA layer. Direct selected-definition prompts also remain bounded,

making them the relevant non-PRA control. This is evidence of bounded active materialization in the toy implementation, not a production memory or latency claim.

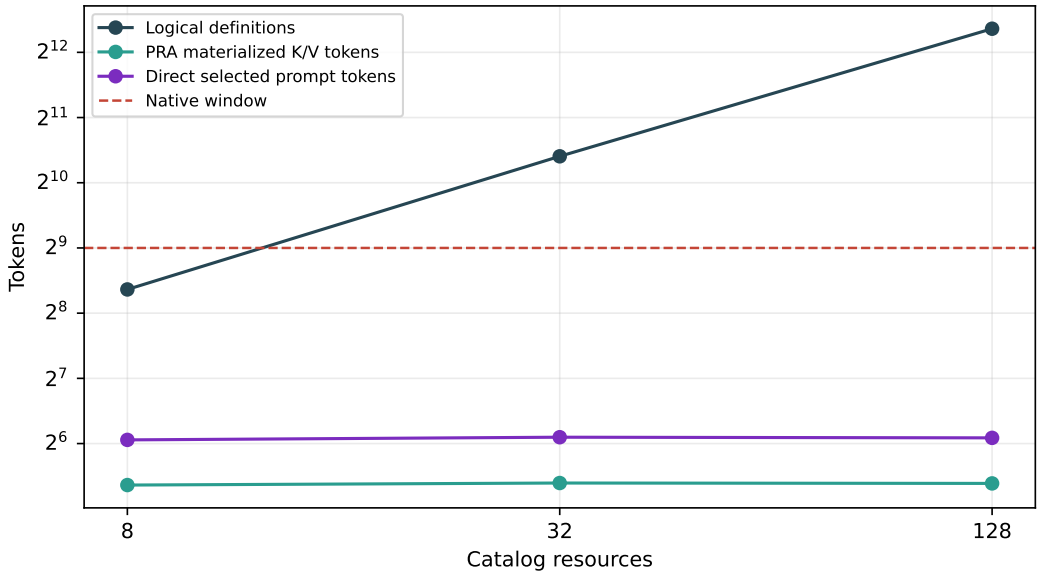

Figure 6: M1 logical catalog text, materialized native K/V per PRA layer, direct selected prompt length, and the 512-token native window. Logical storage and attention-visible working memory are distinct quantities.

The causal result remains narrow. The semantic discoverer is an idealized control, not a learned production index. The grammar is synthetic, the executor is pure and deterministic, and the decoder is trained specifically for this task. The experiment establishes that the selected native K/V carries necessary information through call construction and observation-conditioned continuation; it does not establish general tool use.

8 M2–M4: From Selection to Typed Execution

8.1 Frozen pretrained bridge

M2 freezes Qwen3-0.6B at revision `c1899de289a04d12100db370d81485cdf75e47ca`. Four realistic single-tool tasks cross five deterministic prompt presentations. The model receives OpenAI-compatible function schemas and must emit one parseable `<tool_call>` with the exact function and arguments. Conditions expose the selected tool, the identical oracle tool, a same-family shuffled tool, an irrelevant tool, no tools, or all 18 catalog tools. The five presentations vary instruction wording and eager order; they are not independent model-training seeds.

Selected and oracle disclosure both obtain 1.000 strict success (20/20). Shuffled, irrelevant, and empty conditions obtain 0/20. Eager disclosure obtains 0.850 (17/20). Thus the selected schema causes the output and a small model can use it without task-specific training; the eager catalog is not an oracle and already introduces interference.

8.2 Safe execution and typed observations

M3 inserts a host boundary after generation. The parser treats model JSON as untrusted. The executor checks that the function was disclosed, the stable URI was authorized, required arguments and types match the schema, and write or destructive authority was granted separately. Only registered pure in-memory handlers can run. A result becomes

`!!ref:observation:<namespace>:<call-id>:v1!!`

with producer URI, result schema, call identity, tenant, and provenance. Tool outputs are therefore retrievable resources rather than irreversible transcript text. All 20 selected calls execute and create typed observations. In 17/20 continuations (0.850), the model includes every returned value. The remaining three are consumption failures, despite correct execution.

8.3 Reactive multi-tool composition

M4 uses three held-out workflows of length three, four, and five. Reactive JIT shows only the next discovered schema after each typed result; eager-required shows the complete evaluator-required set at every step; no-refresh leaves only the initial tool visible. The same model and prompt presentations are used. Strict task success requires every expected identity, exact arguments, accepted execution, and typed observation in order.

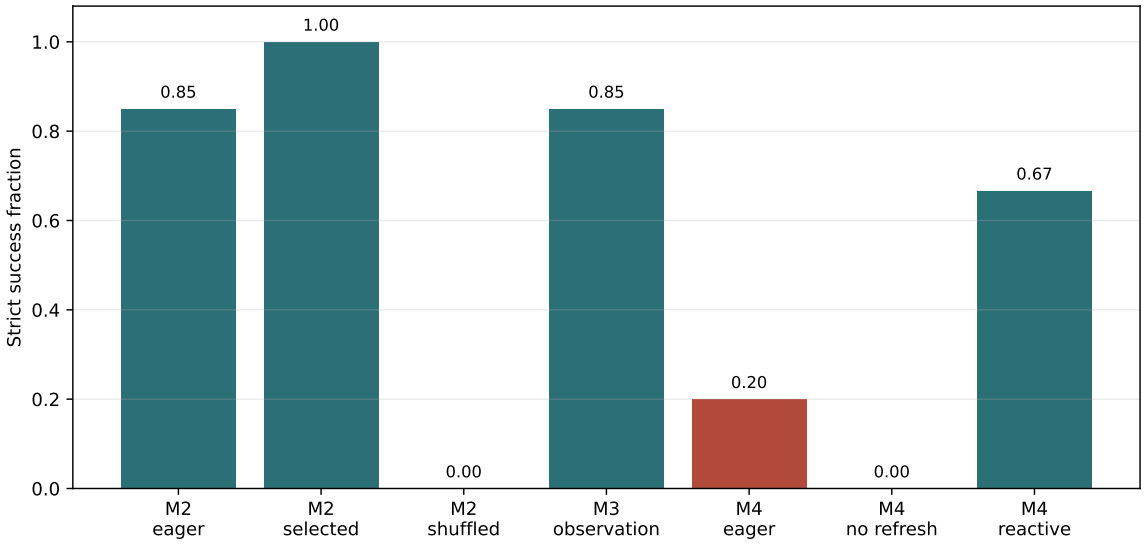

Figure 7: M2–M4 strict gates. Selected definitions cause exact pretrained calls; typed execution is reliable, while observation consumption remains imperfect. Reactive disclosure substantially outperforms both a static required set and a no-refresh control on the small workflow cohort.

Reactive JIT completes 0.667 (10/15), eager-required 0.200 (3/15), and no-refresh 0/15. The three- and four-step families account for the reactive successes; the five-step cross-domain repository workflow fails after its first result in every presentation. M4 is therefore a positive reactive-composition gate, not general long-horizon agent competence.

9 M5: Speculative Capability Disclosure

M5 asks whether exposing a useful capability neighborhood before the first call improves planning relative to direct Top-1 or reactive retrieval. The 18-node graph has 171 typed edges. We compare all-tools eager (P0), direct Top-1 and Top-K (P1–P2), family/category, tag/keyword, and schema expansion (P3–P5), their bounded union (P6), reactive JIT (P7), horizon-matched speculative disclosure (P8), and the exact required set (P9). Labels distinguish required, useful, related-but-unnecessary, irrelevant, and unsafe tools.

Table 3: M5 capability-set and execution results over three workflows. Recall and initial tool counts are averaged across tasks. Model success has 15 trials for P1, P6–P9; a dash denotes a set-only policy.

Policy	Required recall	Initial tools	Unsafe	Model success
P0 all eager	1.000	18.00	1.00	–
P1 direct Top-1	.261	1.00	0	.000
P2 direct Top-K	.822	4.00	0	–
P5 schema graph	.628	3.67	0	–
P6 combined graph	0.933	6.67	0	0.000
P7 reactive JIT	.261	1.00	0	.667
P8 speculative matched	.628	3.67	0	.000
P9 oracle capabilities	1.000	4.00	0	.200

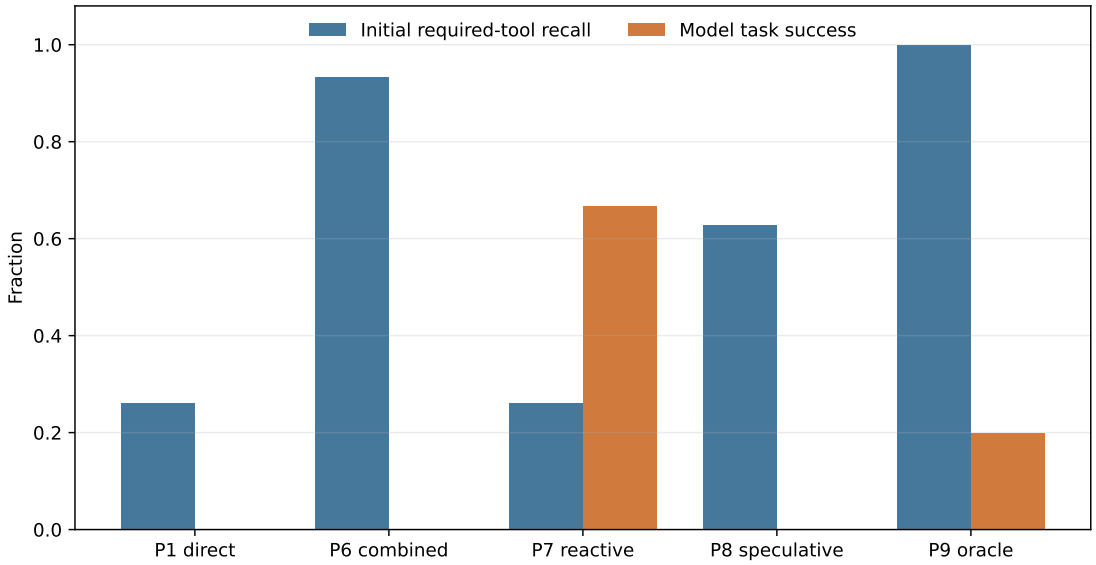

Figure 8: Initial required-tool recall is not task success. The combined graph recovers most required capabilities without unsafe exposure, yet the frozen model does not execute successfully from the larger static set.

The M5 stop/go gate is negative. P6 fully covers two workflows and reaches .933 mean required recall, but static P6 and P8 disclosure obtain 0/15 model success. Even P9 obtains only 3/15, whereas P7 reactive JIT obtains 10/15. The experiment does not support training an adaptive

disclosure controller. The fixed graph remains useful for auditable candidate construction, but a useful capability set expands the action space; it does not solve action sequencing.

10 M6: Optional PRA-Native Resource Discovery

Native Q/K is not treated as another ordinary SDK function. In a generic deployment, the agent and resolver sit outside an opaque model API and cannot read hidden states, native query vectors, key vectors, or PRA cache entries. M6 therefore defines four explicit arrangements: co-located execution; a same-host shared-memory index that exports only $z_q = W_q q$; a model-server resolver endpoint that returns typed URIs and scores; and replicated query computation requiring the same model, tokenizer, layer, projection, and position treatment. Only the co-located mechanism is executed here. The SDK also implements the projected-query payload and identity-only server reply; raw Q/K is not persisted in artifacts.

At frozen Qwen layer 27, 1,625 tokens from 18 structured definitions are encoded with explicit name, description, parameter, return, side-effect, and tag boundaries. Five prompt presentations cover four single-tool and three multi-tool tasks. Baselines include token and indexed lexical routing, a signed-hash control, input-embedding means, native mean K, full token QK, and a lexical/native fusion. The rank-16 and rank-8/eight-centroid projections are the five Paper-2.8 HotpotQA/QASPER checkpoints applied zero-shot; they receive no tool supervision.

Table 4: M6 co-located discovery. Multi-step recall uses a budget equal to the workflow horizon. Bytes are declared persistent routing-index payloads, not Python/process overhead or backing native K/V.

Mode	Single Top-1	Multi recall	Successor recall	Index bytes
Token	.800	.889	1.000	13,427
Index	1.000	.862	1.000	13,427
Signed hash	.800	.670	.906	9,216
Input-embedding mean	.650	.619	.572	36,864
Lexical + rank-16	.800	.690	.689	65,427
Native mean K	0.000	.439	.611	36,864
Native token QK	.000	.439	.611	3,328,000
Paper-2.8 rank-16	.000	0.222	.167	52,000
Paper-2.8 rank-8/8-centroid	.000	.372	.167	2,304

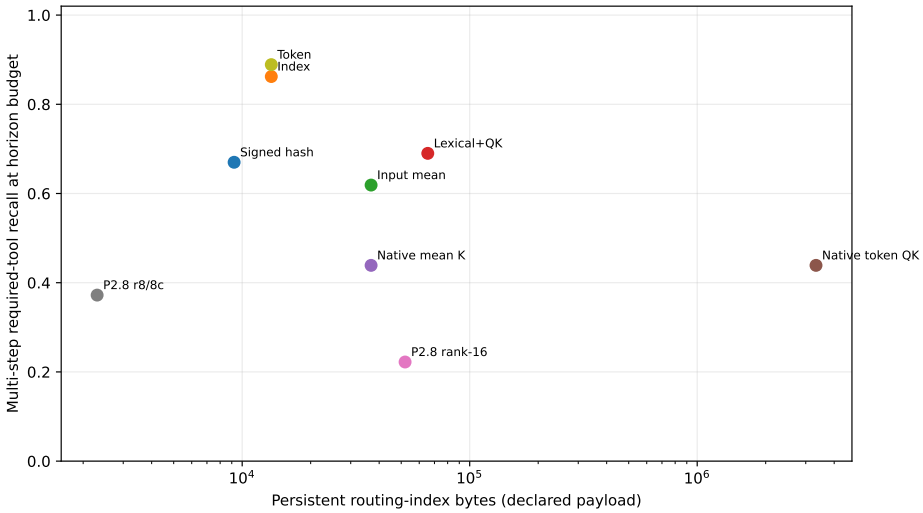

Figure 9: M6 multi-step retrieval quality against routing-index payload. The compact transferred projection is inexpensive, but lexical channels dominate quality on this catalog. Full native-token scoring is both worse and larger.

M6 also stops negatively. Native query/key state does not improve this tool catalog, and the QA-trained low-rank map does not transfer to capability language. The lexical/native fusion is below either lexical channel alone. Consequently the supported generic architecture remains external explicit/token/index discovery. Model-server native routing is an optional research interface whose tighter coupling is not justified by these results.

11 M6.5: External Semantic-Hard Discovery

11.1 Frozen benchmark and split discipline

M6 answers a canonical-language question; it does not show that lexical search is sufficient for sloppy intent. We therefore freeze 306 single-tool requests over the same 18-tool catalog. Each identity has one held-out test and one validation realization in H0 (canonical), H1 (synonym), H2 (colloquial), H3 (implicit goal), H4 (bounded context plus shorthand), and H5 (Portuguese, Spanish, or French against English tool definitions). Eighteen additional audit rows cover the catalog. Validation and test contain 144 identities each; the complete query fingerprint is `e348334ad2362f284a217f1f852f175917a17540dd403f79953d7d2a7ebd7adb`. For H2 and later, tests reject exact tool names and aliases. H4 gives every channel the same context. No test wording selects an encoder, representation, fusion weight, or confidence threshold.

This is a controlled, project-authored fixture rather than a naturally sampled user corpus. Its multilingual mappings and tool tags share the benchmark's typed concept inventory. Their accuracy measures the value of registration-time semantic metadata under those declared assumptions, not open-domain translation or multilingual generalization. The benchmark evaluates discovery only; the multi-step planning and execution boundaries remain those measured in M4–M5.

11.2 External policies

The concept map resolves domain-bounded operation/object phrases and retains language, source, and weight for every expansion. Tool registration emits four representations: description; name plus description; a deterministic card with purpose, operation, object, input/output types, and side effect; and three separate vectors for those fields. Three compact encoders are selected on validation only: all-MiniLM-L6-v2, BGE-small-en-v1.5, and multilingual MiniLM-L12-v2, all pinned by revision. Validation selects BGE with description cards and context-plus-query for English, and multilingual MiniLM with structured cards and query-only encoding for H5. Tool vectors are persisted; third-party weights are not copied into the repository.

The policy ladder compares token overlap (P0), BM25 (P1), typed dictionary expansion (P2), registration-time tags (P3), their lexical combinations, English and multilingual embeddings, a validation-frozen hybrid, and an identity oracle. A staged P10 resolver tries lexical, dictionary/tags, and the embedding hybrid before asking. Its four thresholds are fitted on validation.

Table 5: M6.5 results on 144 frozen test requests. Latency is the median measured end-to-end resolver time for this 18-candidate Python implementation; embedding registration is amortized.

Policy	Top-1	MRR	Recall@3	Median ms
Token	.257	.398	.403	8.6
BM25	.347	.485	.514	8.6
Typed dictionary	.729	.812	.868	8.6
Registration tags	.743	.823	.875	8.6
English embedding	.563	.682	.750	12.3
Multilingual embedding	.556	.690	.785	10.8
Dictionary/embedding hybrid	.729	.809	.875	12.3
Identity oracle	1.000	1.000	1.000	8.6

Tags improve Top-1 over BM25 by .396 (paired identity bootstrap 95% interval [.299, .500]; exact discordance $p < 10^{-10}$). The gain is not uniform. English BGE is strongest on H3 implicit goals (.722), tags on H4 context (.556), and curated tags are perfect on H5 because all three languages are represented in the frozen concept inventory. The hybrid improves H2 to .667 and H5 to .963 but does not dominate each specialized channel. This is why the deployment policy is a calibrated ladder, not a claim that one embedding replaces lexical metadata.

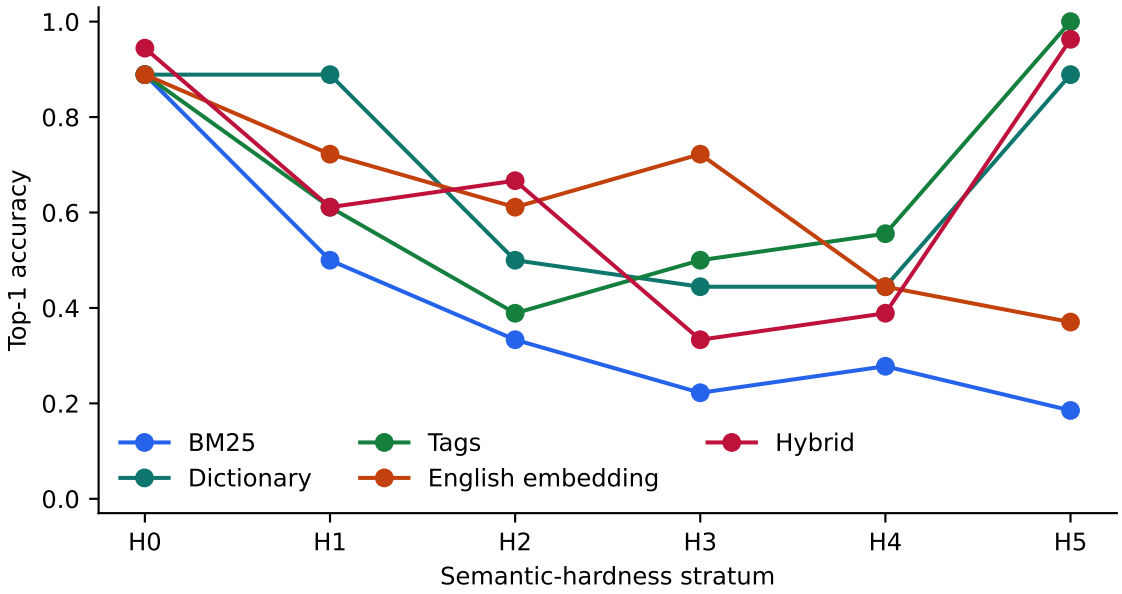

Figure 10: External discovery by hardness. Lexical quality declines as canonical wording disappears; typed metadata and embeddings recover different strata.

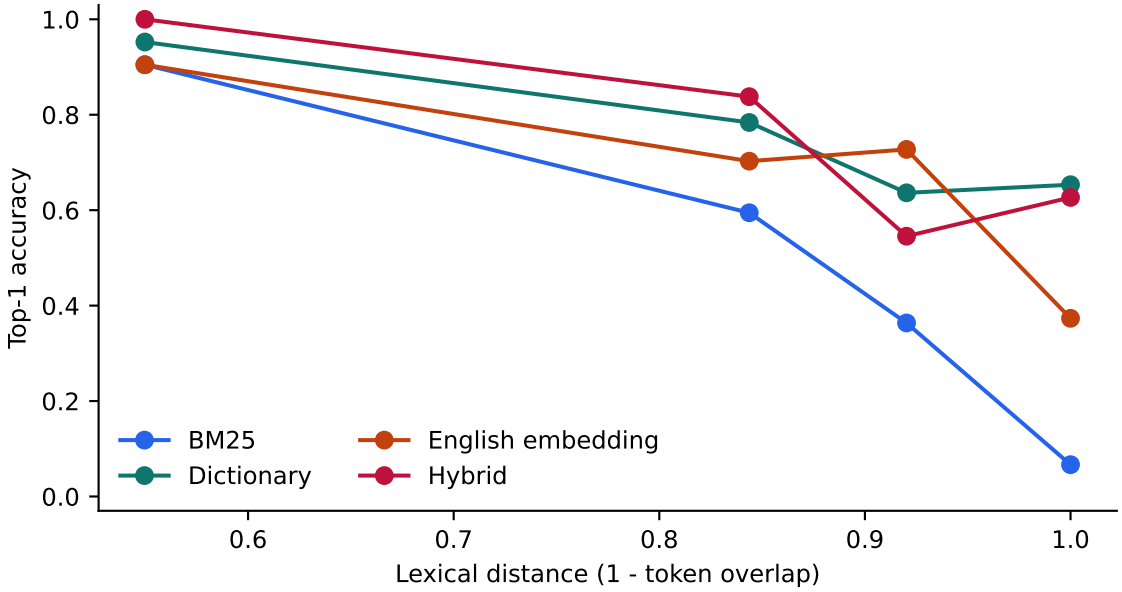

Figure 11: Required quality-versus-distance analysis. Bins contain 21, 37, 11, and 75 test identities from lower to higher lexical distance. Dictionary and hybrid channels retain substantial accuracy when token overlap is zero.

11.3 Selective action and deployment cost

On test, staged P10 selects 86 of 144 requests and asks on 58: coverage is .597, selective accuracy is .907, actionable accuracy is .542, and the false-action rate is .056. No selected top tool is in

the row's unsafe set. Its Brier score is .135 and 10-bin ECE is .098. These held-out numbers are weaker than validation (.949 selective accuracy), which is a reason to retain ASK rather than lower thresholds after seeing test outcomes.

The 384-dimensional persistent tool vectors occupy 27.6 kB. The selected BGE encoder has 133.4 MB of parameters, loads into the process in 3.75 s, and encodes the full 306-query cohort in 1.13 s on the recorded CUDA host; its median per-query resolver path is 12.3 ms versus 8.6 ms for dictionary/tags. The multilingual encoder is larger (470.6 MB) and is not selected for the generic fusion.

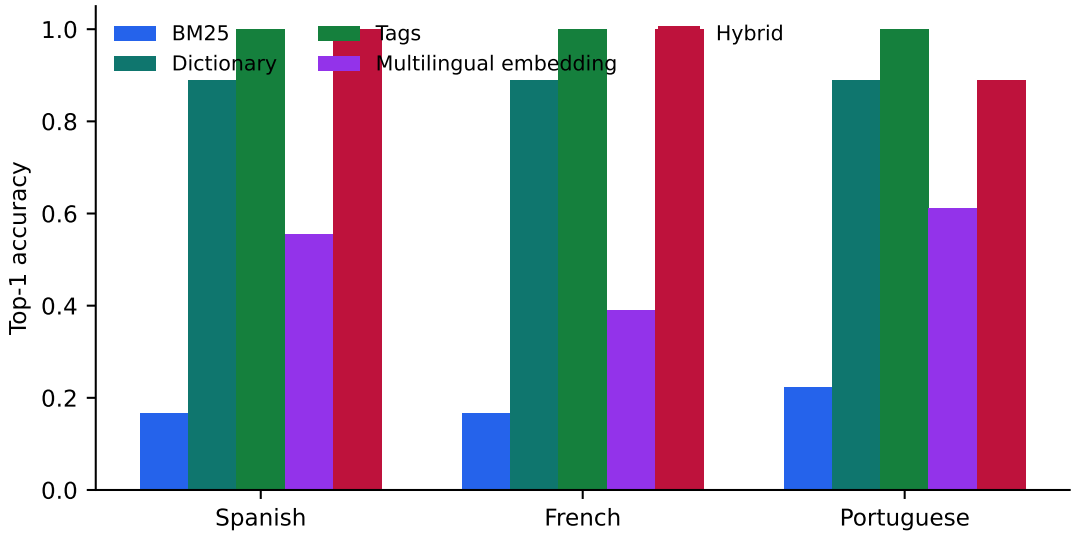

Figure 12: H5 retrieval against English tool definitions. The curated dictionary and tag channels encode the three tested languages by construction; the multilingual encoder is an independent model-backed control.

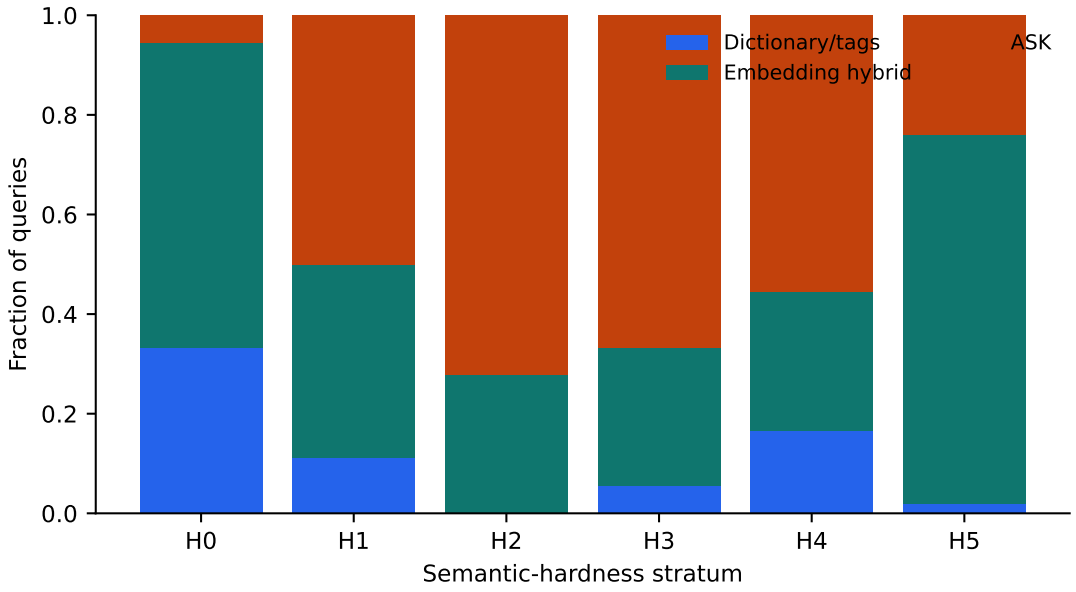

Figure 13: Validation-calibrated P10 stage outcomes on test. More semantic-hard rows reach the hybrid or ASK stages instead of forcing a low-confidence call.

12 M7: Native Geometry on the Frozen Hard Set

M7 reuses all 144 test identities without changing wording, tool cards, or external policies. Frozen Qwen3-0.6B layer-27 pre-RoPE state supplies native mean K and full token QK. The Paper-2.8 rank-16 ensemble and rank-8/eight-centroid ensemble are transferred from five HotpotQA/QASPER checkpoints without tool supervision. Query encoding is charged online; raw native state is not persisted.

Table 6: M7 matched semantic-hard comparison. Model bytes include the frozen Qwen required to reproduce native queries; index bytes exclude backing model weights. The rank-8 timing includes the current per-query centroid construction and is not a production index latency.

Mode	Top-1	MRR	Recall@3	Median ms
BM25	.347	.485	.514	8.6
Tags	.743	.823	.875	8.6
English embedding	.563	.682	.750	12.3
External hybrid	.729	.809	.875	12.3
Native mean K	.056	.233	.313	116.7
Native token QK	.062	.220	.181	115.9
Paper-2.8 rank-16	.056	.194	.167	122.9
Paper-2.8 rank-8/8-centroid	.049	.190	.167	1,658.4

The native Top-1 range, .049–.063, contains the uniform catalog chance rate $1/18 = .056$. Tags exceed native mean K by .688 (paired bootstrap 95% interval [.604,.764]) and full token QK by .681 ([.597,.764]). Thus the harder wording does not reveal a frozen native advantage hidden by M6's canonical queries. This result is narrower than "native Q/K cannot encode tool intent": it says that one frozen layer, two direct reducers, and QA-trained low-rank projections do not expose

it zero-shot for this catalog.

Tool-specific native training does not open. External accuracy leaves oracle headroom, but none of the four frozen native diagnostics supplies the required positive signal, and reproducing native queries requires a 1.19 GB model plus co-location or a model-server extension. The next justified experiment would need tool-supervised evidence independent of this test set, not adaptation to the 144 held-out requests.

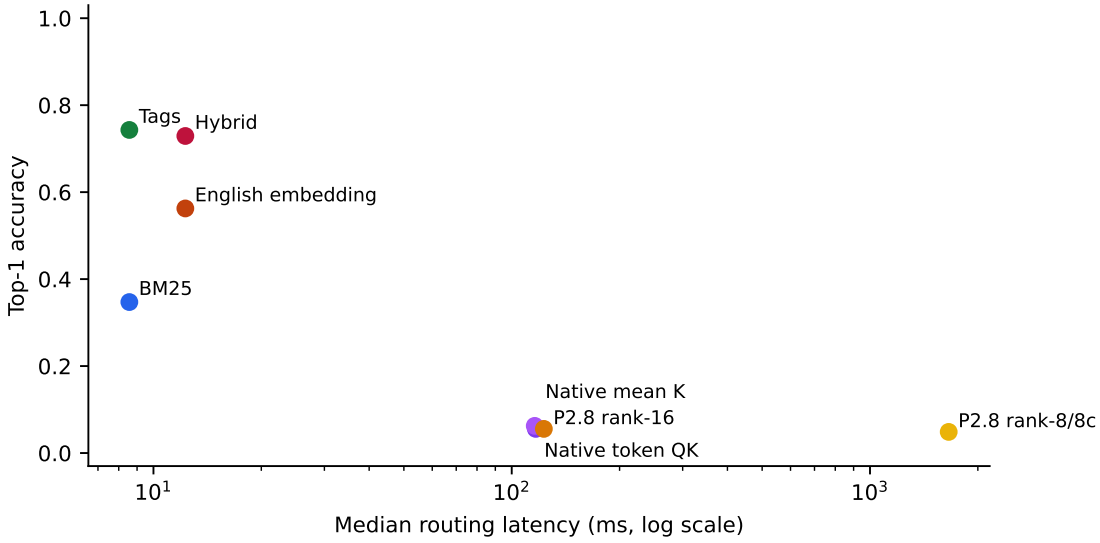

Figure 14: M7 quality–latency frontier on matched queries. External typed semantics dominates the tested frozen native modes in both quality and online latency on this small catalog.

13 Next Experimental Gates

The next gates return to the broader persistent-context thesis: hierarchical skills, mutation and revocation, bounded `#__head` history, superseded facts, and permission-scoped child sessions

$$M_{\text{child}} = R(M_{\text{parent}}) \cup \Delta M_{\text{child}}.$$

Cross-model replication is also required because M2–M7 use one frozen Qwen checkpoint for pre-trained/native mechanisms. Natural semantic-hard requests, larger catalogs, and a tool-identity-disjoint benchmark should test whether the authored tag advantage survives distribution shift. The OpenAI-compatible endpoint belongs after these mechanism checks, so harness behavior does not hide resolver, disclosure, or execution failures.

14 Reproducibility

The M0 benchmark entry points are `experiments/paper6_5_tools/run_m0_policy_study.py` and `summarize_m0_policy_study.py`. Machine-readable outputs include the manifest, one row per policy/query trace, index costs, seed summaries, stratum summaries, oracle distributions, and plots under `docs/papers/shared/results/paper6_5_tools`. The recorded environment is Python 3.10.11 on Windows, Intel64 Family 6 Model 94. All reported M0 results are deterministic conditional on the five catalog seeds.

M1 is reproduced by `run_m1_toy_model.py` followed by `summarize_m1_toy_model.py`. Checked-in artifacts include all 720 evaluation rows, per-step training histories, seed and aggregate summaries, paired effects, a manifest, and generated figures. The run used PyTorch on an NVIDIA GeForce GTX 950M and seeds $\{11, 23, 37, 53, 71\}$. Exact configuration and runtime metadata are recorded in `paper6_5_tools/m1/m1_manifest.json`.

M2–M4 are reproduced by `run_m2_m4_pretrained.py`. M5 uses `run_m5_disclosure.py`; its direct, reactive, and oracle model rows reuse the frozen M4 summaries, while P6/P8 are newly executed static disclosures. M6 uses `run_m6_native_discovery.py`. The final tables, macros, and figures are generated by `summarize_m2_m6.py`. Raw rows contain one call, workflow step, policy/task, or routing-query record as appropriate. Manifests record the frozen model revision, CUDA device, presentation seeds, index accounting, checkpoint provenance, and the fact that all execution handlers are pure in-memory fixtures. M6.5 uses `run_m6_5_external_semantics.py`; M7 uses `run_m7_semantic_native.py`; and `summarize_semantic_hard.py` regenerates paired effects, tables, macros, and figures. Their manifests pin query fingerprints, split discipline, encoder/model revisions, representation selection, thresholds, and CUDA runtime. Tool vectors are committed, but third-party encoder weights and raw native Q/K are not.

15 Conclusion

A large agent catalog should not be mistaken for the context needed on every turn. M0 resolves typed identities without exposing the catalog, and M1 shows that one selected native-K/V definition is used causally. M2–M4 carry the mechanism into a frozen pretrained model: selected schemas support exact calls, host authorization remains separate from generation, typed observations retain provenance, and reactive JIT supports bounded multi-step execution better than static disclosure in this cohort.

The extensions also locate two boundaries. M5 capability graphs improve set recall but not task success; more useful tools visible at once are still more actions for a small model to sequence. M6's canonical queries favor lexical indexing, whereas M6.5 shows that authored tags, dictionaries, and compact embeddings recover semantic-hard intent that BM25 misses. M7 then finds no zero-shot native benefit on those same hard rows. These are positive external- resolver results and bounded native stop results, not evidence that all tool semantics are lexical or that trained native geometry cannot work.

The supported architecture is therefore conservative: explicit identity, indexed lexical search, provenance-bearing dictionary/tags, compact embedding fallback, calibrated ASK, minimal or reactive disclosure, independently authorized execution, and typed observations. This path works with an opaque generation API. Speculative graph neighborhoods remain auditable fixed candidates, and native-Q/K remains an optional model-server research interface. Persistent skills, history, mutation, and session inheritance are the next untested parts of the broader program.